BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State space is the actual world itself a collection of all reachable (x, y) coordinates along with actions that connect them, in this problem at most, width * height states visited once. It is a graph and the same state exists once irrespective of how many ways we have to reach there.

Search tree is the data structure built by the algorithm during its traversal through the search space, a tree whose nodes are actually the paths from the root, and not the state itself. The same state (like cell (3,3)), for example, will occur in many nodes within the search tree depending upon how many paths we had to reach it, since a tree node represents "How did I get here?" and not "Where am I?" This makes the search tree much larger than the state space, and an unbounded search tree loops infinitely even on a finite state space (Up -> Down -> Up -> Down....) unless something prunes the repeated states.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

With reached, we have solved the problem of repetition of states: without it, we do a tree search in the implicit graph of the state space, and because the grid contains cycles (every open state has more than one way to return to it. For example, by going right and left), the algorithm can repeat the same state infinitely many times.

In case of depth-first search, this is a critical issue, rather than an inefficient one. DFS takes a decision and goes along that branch until it reaches a dead end; without reached, "dead end" never occurs in a cycle-containing grid since the agent can simply stay in a couple of neighboring cells indefinitely (right, left, right, left...). Although the number of steps limit (self.steps >= 60) will eventually stop the episode, the algorithm call itself will get stuck if run indefinitely offline (like in dfs_search, when the whole plan needs to be constructed before it returns), thus creating an infinite tree and freezing sense_and_act.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces

winding, suboptimal routes?

Since BFS expands its nodes in order of increasing path length (depth 1 before depth 2 before depth 3, etc.), where the order of nodes is determined by a FIFO (first in, first out) queue, every time BFS expands a node from the frontier, it will always do so based on the shortest possible path, given that all moves have equal cost (exactly 1 in the grid), and thus any deeper node cannot possibly have beaten the node in terms of path length since it wasn't explored yet by BFS.

In contrast to the previous method, DFS uses a LIFO (last in, first out) stack for its frontier, which means that it goes as deep as it can into the last generated branch, without ever taking into account whether it is actually closer to the goal than others. DFS will turn around once it reaches a dead end (which is a wall, boundary, or a previously reached state). Thus, the first solution it finds will simply depend on which path it has decided to commit to first; it may be the shortest, but there is nothing ensuring this fact in general.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

The problem with BFS and UCS is that both algorithms require storing their entire frontier in memory, that is, all nodes of the current depth level, which needs to be completed before going on to the next level. At depth d, the frontier size approaches the size of the reachable state space (up to ~1,000,000 cells) for a 1000x1000 grid in case when food is far from the starting point. The reached set alone contains up to a million (x,y) pairs, and the frontier queue is roughly the same size. This is what causes the crash – not the time taken by the search, but the memory usage of the frontier and reached set.

DFS has the linear in depth memory limit, O(b·m), since DFS only needs to store the current path it traverses (and its unexplored siblings). This means that DFS could traverse the entire 1000x1000 grid with relatively modest amounts of memory, although – according to Q3, this traversal would produce an overly long nonoptimal path, whose time complexity in the worst case scenario would still be exponential in case if m >> d.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

FACULTY OF COMPUTING